
Redes Neurais e Aprendizagem Profunda

Arquiteturas CNN

Sumário

Arquiteturas CNN

Modelos pré-treinados

Arquitetura RNC

LeNet-5

- Yann LeCun, 1998
- Diferenças principais para CNN modernas
 - RELU em vez de tanh
 - softmax em vez de RBF
- Demo na classificação de algarismos
 - <http://yann.lecun.com/exdb/lenet>

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected	–	10	–	–	RBF
F6	Fully connected	–	84	–	–	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	–	–	–

Demonstração: <http://yann.lecun.com/exdb/lenet/index.html>

AlexNet

- **Alex** Krizhevsky, Ilya Sutskever, Geoffrey Hinton, 2012
- Vencedor do ILSVRC
 - erro top-5: 17% (segundo melhor: 26%)
- Semelhante a LeNet-5
 - muito maior e profunda
 - várias camadas convolucionais umas em cima das outras
- Regularização
 - dropout rate: 50% à saída de F9 e F10
 - **aumento de dados** (*data augmentation*)
 - deslocamento, flip horizontal, alteração das condições de iluminação
- Normalização
 - *local response normalization* (LNR) depois do ReLU nas camadas C1 e C3

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected	–	1,000	–	–	–	Softmax
F9	Fully connected	–	4,096	–	–	–	ReLU
F8	Fully connected	–	4,096	–	–	–	ReLU
C7	Convolution	256	13×13	3×3	1	same	ReLU
C6	Convolution	384	13×13	3×3	1	same	ReLU
C5	Convolution	384	13×13	3×3	1	same	ReLU
S4	Max pooling	256	13×13	3×3	2	valid	–
C3	Convolution	256	27×27	5×5	1	same	ReLU
S2	Max pooling	96	27×27	3×3	2	valid	–
C1	Convolution	96	55×55	11×11	4	valid	ReLU
In	Input	3 (RGB)	227×227	–	–	–	–

Local response normalization (LNR)

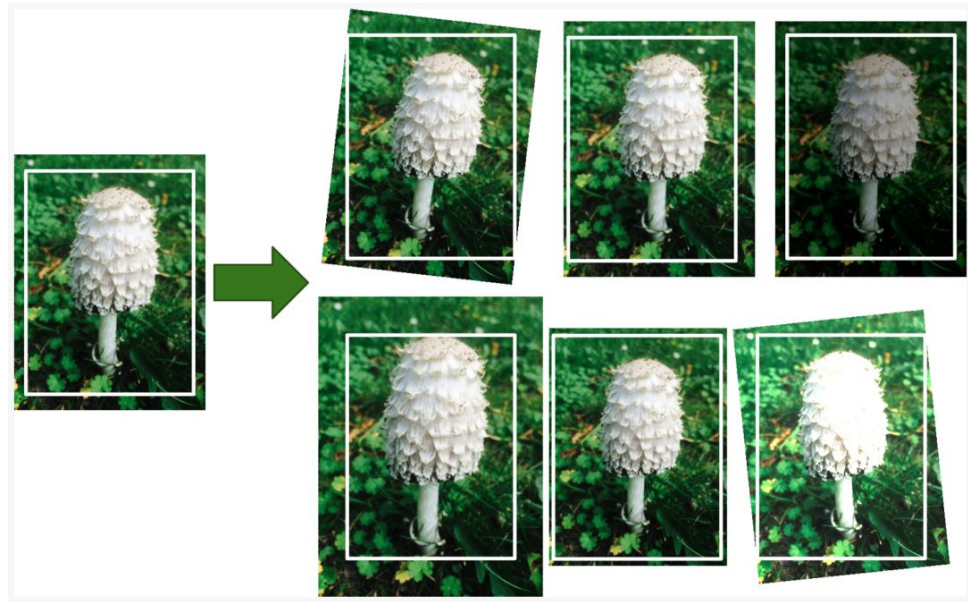
- Funcionamento
 - neurónios com ativação mais forte inibem outros neurónios na mesma posição em mapas da vizinhança
- Melhora a generalização
 - encoraja a especialização dos mapas, distanciando-os e forçando a exploração de maior variedade de características

$$b_i = a_i \left(k + \alpha \sum_{j=j_{\text{low}}}^{j_{\text{high}}} a_j^2 \right)^{-\beta} \quad \text{with} \quad \begin{cases} j_{\text{high}} = \min \left(i + \frac{r}{2}, f_n - 1 \right) \\ j_{\text{low}} = \max \left(0, i - \frac{r}{2} \right) \end{cases}$$

- b_i
 - saída normalizada do neurónio do mapa i (numa linha u , coluna v)
- a_i
 - ativação do neurónio após ReLU (mas antes da normalização)
- f_n
 - número de mapas
- k, α, β, r
 - hiper-parâmetros
 - k : bias, r : depth radius
- AlexNet
 - $r=5, \alpha=0.0001, \beta=0.75, k=2$

Aumento de dados

- Aumento artificial do conj. de treino
 - geração realista de variantes de cada exemplo
 - humano incapaz de diferenciar se foi gerada ou não
- Exemplos
 - fracas deslocação, rotação, flip, re-dimensionamento (c/ diferentes valores)
- Técnica de regularização
 - obriga o modelo a ser mais tolerante a pequenas variações de posição, orientação, tamanho, condições de iluminação



ZF Net

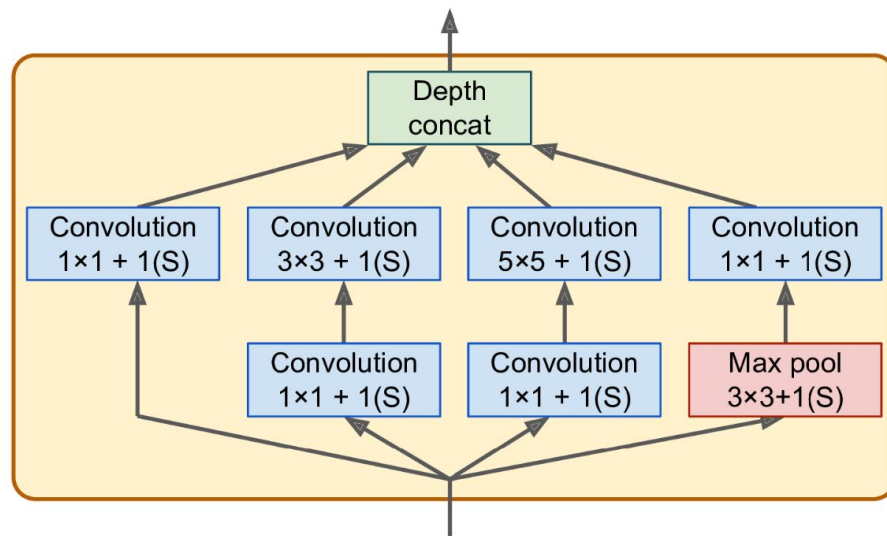
- Matthew Zeiler, Rob Fergus, 2013
- Vencedor do ILSVRC
- AlexNet com hiper-parâmetros afinados
 - nº de mapas, tamanho do kernel, stride, etc

GoogLeNet

- Christian Szegedy et al, 2014
 - Google Research
- Vencedor do ILSVRC
 - erro top-5: abaixo de 7%
- Módulo inception
 - sub-rede
 - permite arquitetura muito mais profunda
 - ... mas com 10x menos parâmetros que AlexNet
 - 60 milhões -> 6 milhões
 - capaz de gerar mapas que capturam padrões complexos em várias escalas

Módulo inception

- sinal copiado para 4 camadas distintas
- camadas convolucionais
 - função de ativação ReLU
- saídas têm o mesmo tamanho das entradas (altura e largura)
 - stride=1. padding=same
- *depth concatenation layer*
 - empilha os mapas das 4 camadas convolucionais do topo



notação

- $3 \times 3 + 1(S)$: kernel 3×3 , stride=1, padding=same

Porquê camada com kernel 1x1?

- não captura padrões espaciais mas captura padrões em profundidade
- funciona como camada de bottleneck
 - configurada para gerar menos mapas de saída (que o nº de mapas de entrada)
 - diminui o custo computacional e nº de parâmetros -> treino mais rápido e melhor generalização
- Cada par de camadas [1x1, 3x3] e [1x1, 5x5]
 - age como uma única camada convolucional poderosa capaz de capturar padrões mais complexos

VGGNet

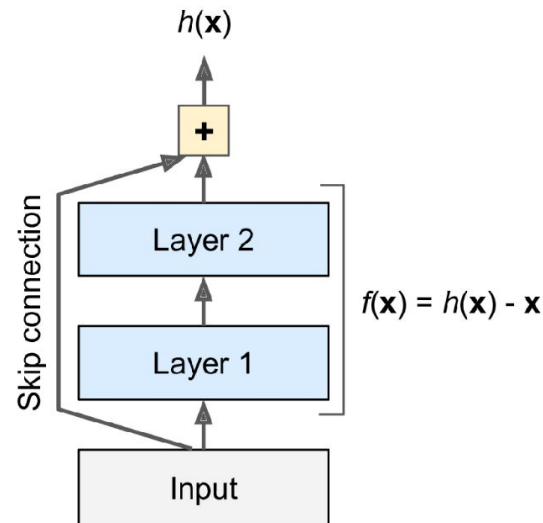
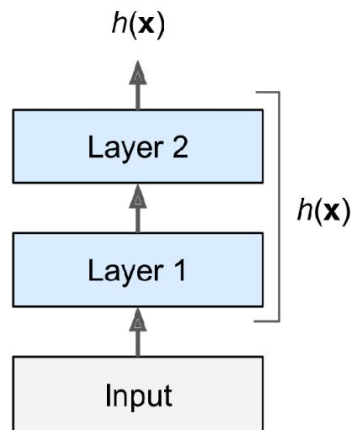
- Karen Simonyan, Andrew Zisserman, 2014
 - Visual Geometry Group (VGG) research lab at Oxford University
- Vice-campeão do ILSVRC
- Arquitetura simples e clássica
 - 2 ou 3 camadas convolucionais + camada de pooling
 - filtros 3x3, muitos filtros
 - repetição até um total de 16 ou 19 camadas convolucionais (dependendo da variante)
 - rede densa com 2 camadas escondidas
 - camada de saída

ResNet

- Kaiming He et al, 2015
- Vencedor do ILSVRC
 - erro top-5: abaixo de 3.6%
- Rede Residual
 - 152 camadas
 - variantes com 24, 50 e 101 camadas
 - utiliza *skip connections* (ou *shortcut connections*)
 - o sinal de entrada de uma camada é também adicionado à saída de uma camada algures superior na pilha

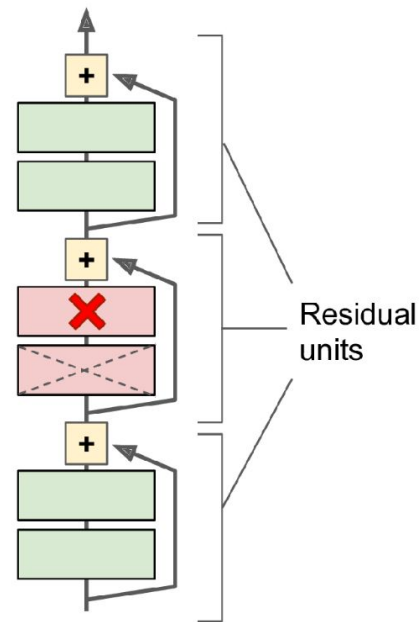
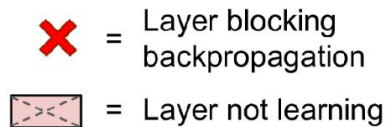
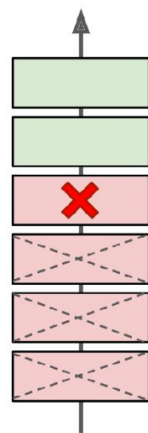
Aprendizagem residual

- Rede neuronal regular
 - objetivo: modelar a função objetivo $h(\mathbf{x})$
 - inicialização: pesos próximo de zero
 - saída próxima de zero
- Adicionando x à saída da rede
 - resultado: rede vai modelar $f(\mathbf{x})=h(\mathbf{x})-\mathbf{x}$
 - inicialização
 - saída é cópia da entrada
 - modela função identidade
 - se a função for próxima da identidade
 - treino muito mais rápido



Rede residual profunda

- É uma pilha de unidades residuais
 - a rede faz progressos mesmo quando algumas camadas ainda não começaram a aprender
- Unidade residual
 - rede neuronal pequena com uma skip connection

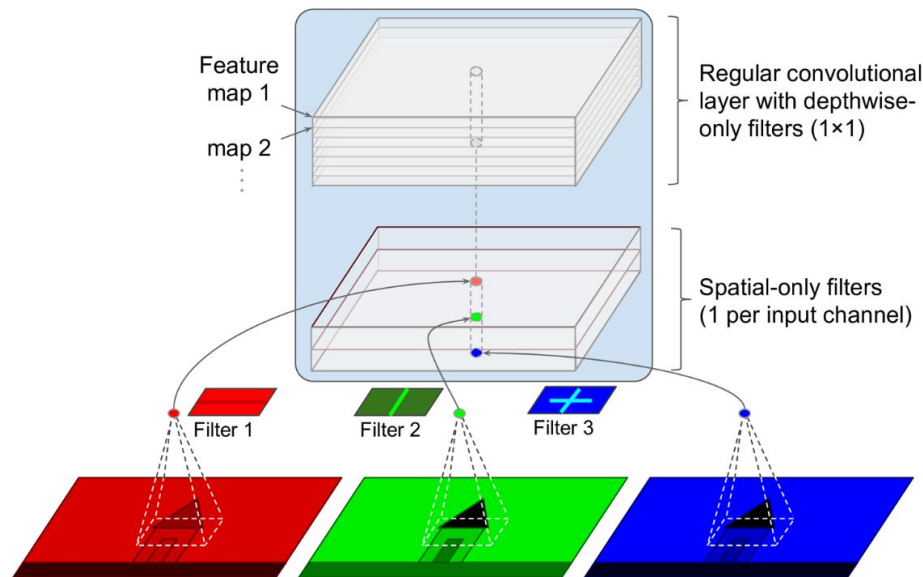


Xception (Extreme Inception)

- François Chollet, 2016
- Junta as ideias de GoogLeNet e ResNet
 - substitui os módulos inception por uma **camada convolucional depthwise separable**
- Camada convolucional tradicional
 - tenta capturar simultaneamente padrões espaciais (ex: oval) e entre canais (ex: boca+nariz+olhos=face)
- Camada convolucional depthwise separable
 - suposição forte que padrões espaciais e entre canais podem ser modelados separadamente
- Arquitetura
 - 2 camadas convolucionais regulares
 - 34 camadas convolucionais depthwise separable
 - algumas camadas max-pooling
 - camada global average pooling
 - camada densa

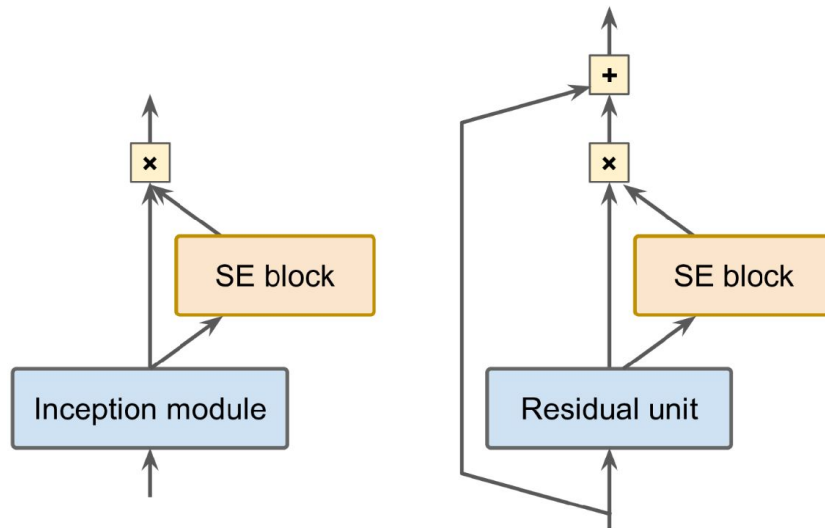
Camada convolucional depthwise separable

- Composta por 2 partes
 - parte 1: um filtro espacial para cada mapa
 - parte 2: uma camada convolucional com kernel 1×1
 - olha exclusivamente para padrões entre canais
- Evitar usar depois de camadas com poucos canais
- Características
 - menos parâmetros, menos memória que camada convolucional regular
 - têm melhor desempenho
 - deve ser **usada por omissão**



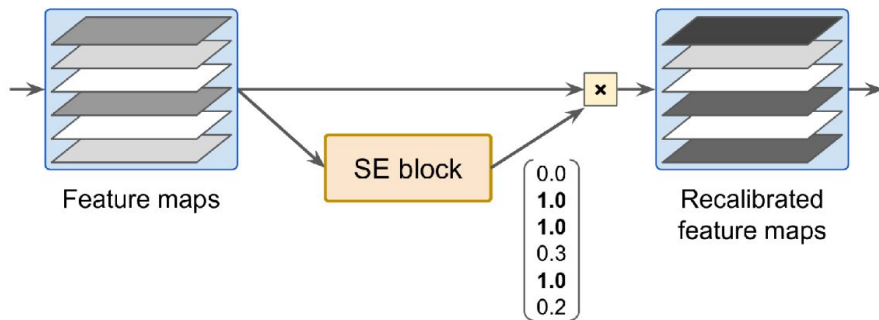
SENet (Squeeze-and-Excitation Network)

- Jie Hu et al, 2017
- Vencedor do ILSVRC
 - erro top-5: 2.25%
- Estende as ideias das redes inception e ResNet
 - Usa **bloco SE**
 - SE-Inception
 - adiciona um bloco SE a cada módulo inception
 - SE-ResNet
 - adiciona um bloco SE a cada unidade residual



Funcionamento do bloco SE

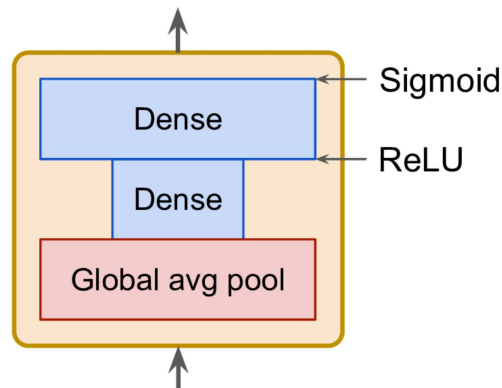
- Analisa a saída da unidade a que está ligada focando exclusivamente na profundidade
- Aprende que características estão ativas ao mesmo tempo
- Usa esta informação para recalibrar os mapas



- Exemplo
 - Bloco SE aprende que boca, nariz, olhos aparecem usualmente nas imagens
 - Se observar uma ativação forte para boca e nariz e fraca para olhos, vai impulsionar esse mapa
 - Se os olhos foram confundidos com outra coisa, esta recalibragem ajuda a resolver a ambiguidade

Bloco SE

- camada pooling média global
 - calcula a média de ativação para cada mapa
 - ex: 256 mapas -> 256 números
- camada escondida densa com ativação ReLU
 - **squeeze**: tipicamente 16 vezes menos neurónios que o nº de mapas
 - representação vetorial de baixa dimensão (embedding) da distribuição das respostas das características
 - força a aprender uma representação da combinação de características
- camada de saída densa com ativação sigmoide
 - produz um vetor de recalibração com valor entre 0 e 1 por mapa a partir do embedding
 - os mapas são multiplicados pelo vetor de recalibração
 - características irrelevantes (valor próximo de zero) ficam com peso menor
 - características relevantes (valor próximo de 1) mantêm o peso



Outras arquiteturas

- ResNeXt
 - <https://arxiv.org/abs/1611.05431>
- DenseNet
 - <https://arxiv.org/abs/1608.06993>
- MobileNet
 - <https://arxiv.org/abs/1704.04861>
- CSPNet
 - <https://arxiv.org/abs/1911.11929>
- EfficientNet
 - <https://arxiv.org/abs/1905.11946>

Escolher a arquitetura

- Depende do que é importante
 - desempenho
 - tamanho do modelo
 - velocidade de inferência
 - CPU or GPU
- Modelos pré-treinados no keras
 - <https://keras.io/api/applications>

Class name	Size (MB)	Top-1 acc	Top-5 acc	Params	CPU (ms)	GPU (ms)
MobileNetV2	14	71.3%	90.1%	3.5M	25.9	3.8
MobileNet	16	70.4%	89.5%	4.3M	22.6	3.4
NASNetMobile	23	74.4%	91.9%	5.3M	27.0	6.7
EfficientNetB0	29	77.1%	93.3%	5.3M	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	140.0	8.8
EfficientNetB4	75	82.9%	96.4%	19.5M	308.3	15.1
InceptionV3	92	77.9%	93.7%	23.9M	42.2	6.9
ResNet50V2	98	76.0%	93.0%	25.6M	45.6	4.4
EfficientNetB5	118	83.6%	96.7%	30.6M	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	958.1	40.4
ResNet101V2	171	77.2%	93.8%	44.7M	72.7	5.4
InceptionResNetV2	215	80.3%	95.3%	55.9M	130.2	10.0
EfficientNetB7	256	84.3%	97.0%	66.7M	1578.9	61.6

inference time on batches of 32 images

92-core AMD EPYC CPU with IBPB, 1.7 TB of RAM, and an Nvidia Tesla A100 GPU.

ResNet-34 com Keras

```
class ResidualUnit(keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = keras.activations.get(activation)
        self.main_layers = [
            keras.layers.Conv2D(filters, 3, strides=strides,
                                padding="same", use_bias=False),
            keras.layers.BatchNormalization(),
            self.activation,
            keras.layers.Conv2D(filters, 3, strides=1,
                                padding="same", use_bias=False),
            keras.layers.BatchNormalization()]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                keras.layers.Conv2D(filters, 1, strides=strides,
                                    padding="same", use_bias=False),
                keras.layers.BatchNormalization()]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers:
            Z = layer(Z)
        skip_Z = inputs
        for layer in self.skip_layers:
            skip_Z = layer(skip_Z)
        return self.activation(Z + skip_Z)
```

```
model = keras.models.Sequential()
model.add(keras.layers.Conv2D(64, 7, strides=2, input_shape=[224, 224, 3],
                              padding="same", use_bias=False))
model.add(keras.layers.BatchNormalization())
model.add(keras.layers.Activation("relu"))
model.add(keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"))
prev_filters = 64
for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
    strides = 1 if filters == prev_filters else 2
    model.add(ResidualUnit(filters, strides=strides))
    prev_filters = filters
model.add(keras.layers.GlobalAvgPool2D())
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(10, activation="softmax"))
```

Keras tem várias arquiteturas integradas!

Modelos pré-treinados

Pacote `tf.keras.applications`

- Contém várias redes pré-treinadas
 - mais de 35 modelos disponíveis
 - <https://keras.io/api/applications/>
- Exemplo
 - modelo ResNet50 pré-treinado com ImageNet

```
model = tf.keras.applications.ResNet50(weights="imagenet")
```

- Antes de usar
 - assegurar que as imagens têm o tamanho adequado
 - `tf.keras.layers.Resizing()`
 - verificar se as entradas necessitam ser escaladas
 - `preprocess_input()`
 - função existente em cada modelo
 - assume que os valores dos pixels variam entre 0 e 255

Exemplo

```
model = tf.keras.applications.ResNet50(weights="imagenet")

images = load_sample_images()["images"]
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                           crop_to_aspect_ratio=True)(images)

inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)

Y_proba = model.predict(inputs)

top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f"  {class_id} - {name:12s} {y_proba:.2%}")
```

mostra o nome da classe e a respectiva
estimação de probabilidade

Image #0		
n03877845	- palace	54.69%
n03781244	- monastery	24.72%
n02825657	- bell_cote	18.55%
Image #1		
n04522168	- vase	32.66%
n11939491	- daisy	17.81%
n03530642	- honeycomb	12.06%

Transfer Learning

Criar um modelo para classificar flores, reutilizando o modelo Xception

```
import tensorflow_datasets as tfds
```

```
dataset, info = tfds.load("tf_flowers", as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples # 3670
class_names = info.features["label"].names # ["dandelion", "daisy", ...]
n_classes = info.features["label"].num_classes # 5
```

carregar o conjunto

```
test_set_raw, valid_set_raw, train_set_raw = tfds.load(
    "tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],
    as_supervised=True)
```

criar os conjuntos treino, validação e teste

```
batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224, crop_to_aspect_ratio=True),
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])
train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
```

definir o batch
garantir que todas as imagens têm o mesmo tamanho
baralhar o conjunto treino e fazer pré-fetch

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)
])
```

aumentar os dados

```
base_model = tf.keras.applications.xception.Xception(weights="imagenet",
                                                    include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)
```

carregar o modelo Xception, excluindo as
camadas superiores
adicionar camadas
criar o modelo

```
for layer in base_model.layers:
    layer.trainable = False
```

congelar as camadas no início do treino

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=3)
```

compilar o modelo e começar a treinar (3
épocas)

```
for layer in base_model.layers[56:]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,
              metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=10)
```

quando a exatidão do conj de validação
estagnar, descongelar as camadas e
continuar a treinar

Bibliografia

Capítulo 14

*Hands-On Machine Learning with
Scikit-Learn, Keras & TensorFlow*
